GUIA DE OPERAÇÃO E REFERÊNCIA TÉCNICA

SISTEMA ESFERA

Manual completo do pêndulo esférico controlado: hardware e conexões, firmware do STM32, referência de comandos, calibrações (wizard por motor), procedimentos de bancada e o digital twin em tempo real.

Equipe Esfera · firmware v2 · repositório: commit 8e434ea

Sumário

1. Números-chave **3**

2. Arquitetura e referenciais **3**

2.1 Quem conversa com quem . . . 3

2.2 Referenciais e convenção de eixos . . . 4

2.3 Paridade simulador ↔ firmware ↔ twin . . . 4

3. Hardware e conexões físicas **5**

3.1 Mapa de pinos (Black Pill STM32F401/F411) . . . 5

3.2 Dispositivos . . . 5

3.3 Raspberry Pi . . . 6

4. O firmware: o que roda no STM **6**

4.1 Fusão de atitude (inclinação e heading) . . . 6

4.2 Modelo por motor e alocação . . . 7

4.3 Loop de translação (X e Y): amortecimento . . . 7

4.4 Loop de heading: cascata com frenagem . . . 7

4.5 Segurança . . . 7

5. Referência de comandos **7**

5.1 Comandos de caractere (sem Enter) . . . 7

5.2 Comandos em linha (com Enter) . . . 8

5.3 Índices de ganho (G e R) . . . 8

5.4 Telemetria (25 Hz, as duas portas) . . . 9

5.5 Teclas do simulador interativo . . . 9

5.6 Teclas do twin — semântica dupla . . . 9

5.7 Linhas de comando (PC) . . . 10

6. Procedimento de operação **10**

6.1 Sequência de bancada (com frequência/gatilho de cada etapa) . . . 10

6.2 Fluxo comentado da primeira sessão . . . 10

6.3 Roadmap: fase drone (não implementado) . . . 11

7. Calibrações: automático vs. input do usuário **11**

7.1 Totalmente automático (sem ação do usuário) . . . 11

7.2 Semiautomáticas (você dispara, o sistema confere) . . . 11

7.3 Input do usuário . . . 12

7.4 Ainda não calibrado (a medir na bancada) . . . 12

8. O wizard de identificação por motor 13

8.1 Procedimento (visão do operador) . . . 13

8.2 A matemática, em quatro blocos . . . 13

8.2.1 Subtração do resíduo de oscilação . . . 13

8.2.2 Transporte para o corpo, amostra a amostra . . . 13

8.2.3 Força a partir da resposta do pêndulo . . . 13

8.2.4 Regressão da curva . . . 14

8.3 Saída e validação . . . 14

8.4 Limites . . . 14

9. Digital Twin 14

9.1 Transportes e como escolher . . . 14

9.2 Deploy na Raspberry Pi . . . 15

9.3 A janela do twin . . . 15

9.4 O contrato da visão (IMPLEMENTADO no app da Rasp) . . . 16

10. Configurações avançadas 16

10.1 Constantes do firmware (topo de firmware/main.cpp) . . . 16

10.2 Orçamento de µs por motor . . . 16

10.3 Rescale de ganhos por inércia . . . 17

10.4 Limites de segurança vigentes . . . 17

10.5 PROPOSTA (não implementado): política de perto-do-neutro . . . 17

11. Solução de problemas 17

Apêndice A — Cartão de referência rápida 18

Apêndice B — Fórmulas do modelo 19

Apêndice C — O loop fechado em detalhe 20

1. Números-chave

Valores vigentes no repositório (commit **8e434ea**). Parâmetros marcados como provisórios são substituídos pela calibração do wizard.

Parâmetro	Valor	Origem/estado
Massa total do corpo móvel	12 kg	medido
Raio da esfera	0,60 m (diâmetro 1,20 m)	conforme hardware
Braço eixo → propulsores	0,70 m	conforme hardware
Inércia de heading I_{ψ}	$(2/5)\cdot m\cdot r^2 = 1{,}728\ kg\cdot m^2$	estimativa — calibrar
Comprimento do cabo L	1 m nominal (0,5–6 m)	mudar só entre sessões
Torção do cabo	0 (heading livre, por giroscópio)	conforme hardware
Força por hélice @ ±500 µs	18 N (curva quadrática)	provisório — wizard calibra
Eficiência de ré / zona morta	60% / 30 µs	provisórios — wizard calibra
Faixa de PWM	1500 ± 500 µs (1000–2000)	faixa completa do ESC
Travamento direcional do ESC (latch)	~±60 µs (PWM ~1440–1560)	provisório — wizard medirá por ESC
Reservas de µs (balanço / yaw)	490 / 300	orçamento por motor
Envelope de força líquida	≈ 15,5 N (13% do peso)	derivado da curva
Ganhos XY (kp / ki)	45 N·s/m / 2,2 N·s/m	sintonizados no simulador
Ganhos de heading (perfil / taxa)	decel 1,5 rad/s^2; lim 2,5 rad/s; kp 900 µs; ki 2000 µs	sintonizados
Malha de controle / telemetria	200 Hz / 25 Hz	firmware
Atraso de atuação modelado	120 ms (1ª ordem)	estimativa — medir na bancada

Tabela 1.1 — Parâmetros do sistema.

2. Arquitetura e referenciais

2.1 Quem conversa com quem



Figura 2.1 — Topologia em campo: o app de visão da Rasp é o único dono da serial USB do STM e serve a ponte TCP 9999; a bancada dispensa a Rasp (STM na USB do PC).

Em campo, o STM conversa pela **USB com a Raspberry Pi** — o **app de visão** (Raspberry/main.py) é o único dono dessa serial: processa as câmeras estéreo, detecta o cabo da linha, alimenta o HUD local e envia a visão ao STM (contrato V, Cap. 9.4). Embutida no app roda a **ponte TCP** (Raspberry/bridge.py, porta 9999): tudo o que o STM transmite vai ao twin/wizard no PC, e os comandos deles voltam pela mesma serial. O app também cuida do **reset físico do STM** (GPIO18, executado no boot limpo antes de abrir a serial) e lê o **rádio RC** (canais CH12–15 → pinça/polia).

Na bancada de flash/testes a Rasp não é necessária: o STM pluga na USB do PC e twin/wizard usam --serial. A Serial2 (PA2/PA3) permanece configurada no firmware como link alternativo. No terminal da própria Rasp, a linha de comandos do STM segue disponível através da ponte: nc localhost 9999. Tudo que importa em operação é ajustável sem reflashing — ganhos (G/R), calibração por motor (M, via wizard --push), L da sessão, heading e visão; flash só para mudar lei de controle ou constantes compiladas. Para o primeiro bring-up, siga docs/BRINGUP.md (etapas com motores por último).

2.2 Referenciais e convenção de eixos

- **Corpo** = referencial do sensor (MPU6050 + câmeras estéreo), que gira com o heading. É o referencial em que os empuxos dos motores são definidos.
- **Mundo** = referencial fixo vertical. A força de controle é calculada aqui e girada para o corpo por $R_z(-\psi)$ antes da alocação nos motores.
- Motor **a** (BF, pino PA0) empurra no **+X do sensor** — verificado fisicamente. Motores **b** (AS, PA15) e **c** (TR, PA1) ficam a +120° e −120°: $t_a = (1, 0)$, $t_b = (-1/2, +\sqrt{3}/2)$, $t_c = (-1/2, -\sqrt{3}/2)$.
- Heading ψ cresce no sentido anti-horário visto de cima; o erro de apontamento vem da visão (bearing do cabo) ou do setpoint interno vs. yaw integrado do giroscópio.

2.3 Paridade simulador ↔ firmware ↔ twin

Simulador Python, firmware C++ e STM simulado usam o **mesmo modelo de atuador** (curva por motor, reservas, clamps), os **mesmos ganhos** (mesmos nomes e valores — o heading trabalha em µs nos dois lados) e o **mesmo protocolo**. Consequência prática: um ganho medido no twin vale no firmware, e uma calibração (calib.json) serve para os três. A troca de ganhos pelo twin usa o mesmo comando G que você digitaria na serial.

3. Hardware e conexões físicas

3.1 Mapa de pinos (Black Pill STM32F401/F411)

Pino	Função	Dispositivo / Observação
PA0	ESC motor a (BF)	braço superior (frente); empurra +X do sensor
PA15	ESC motor b (AS)	inferior esquerdo (+120°)
PA1	ESC motor c (TR)	inferior direito (−120°)
PB5 / PB6	POLIA LPWM / RPWM	BTS7960 #1 — polia (varre L)
PB7 / PB10	POLIA EN_R / EN_L	enable da ponte
PB0 / PA7	PINCA LPWM / RPWM	BTS7960 #2 — pinça
PB12 / PB13	PINCA EN_R / EN_L	enable da ponte
PB14 / PB15	fim de curso cima / baixo	INPUT_PULLDOWN; ativo em HIGH
PA5 / PA6	sensor óptico do olhal D0 / A0	LM393; D0 com debounce de 100 ms
PB8 / PB9	I2C1 SCL / SDA	MPU6050 (0x68) + AS5600, 400 kHz
PA2 / PA3	Serial2 TX / RX	115200 baud → Raspberry Pi
PC13	LED onboard	ativo em LOW; pisca na calibração, apaga com erro de yaw

Tabela 3.1 — Pinout do firmware (idêntico ao firmware original nos mecanismos).

3.2 Dispositivos

• **ESCs F45A bidirecionais**: PWM de 1500 µs = parado; 1000/2000 µs = full speed em cada sentido; curva de empuxo quadrática com zona morta. Calibração de faixa já feita; verificar corrente/térmica com a faixa completa (±500 µs) liberada.

• **MPU6050 no topo, junto ao pino**: nessa posição o acelerômetro lê praticamente só a gravidade no referencial do corpo — um inclinômetro quase ideal, sem a corrupção da dinâmica do pêndulo. Giroscópio ±250 °/s, acelerômetro ±2 g, barramento a 400 kHz.

• **AS5600**: encoder magnético do mecanismo da polia (comando 'e' lê o ângulo).

• **LM393 (olhal)**: sensor óptico que detecta se o olhal está engatado no mecanismo.

• **Fins de curso**: travam a pinça nos extremos (subida bloqueada no topo, descida na base).

• **BTS7960 ×2**: pontes H da polia e da pinça, comandadas por comando serial (não participam do controle automático).

3.3 Raspberry Pi

A Rasp tem SSH e saída de vídeo para o monitor: é o posto de operação em campo. Nela roda o **app de visão** (Raspberry/main.py): duas câmeras USB (720p) com retificação estéreo calibrada, mapa de disparidade SGBM + filtro WLS, detecção do cabo por Hough com seleção por profundidade, HUD com telemetria e teclado local, reset físico do STM (GPIO18), leitura do rádio RC (CH12–15, com debounce e arming) e a ponte TCP embutida para o twin/wizard no PC. O modo mono (uma câmera só) mantém o bearing sem a profundidade D. Dependências em Raspberry/requirements.txt — atenção: o filtro WLS exige o pacote **opencv-contrib**.

4. O firmware: o que roda no STM

Todo o controle acontece numa malha de **200 Hz** (passo de 5 ms). A cada volta: lê o MPU6050, atualiza a fusão de atitude, calcula os comandos dos dois loops, aloca nos três motores e escreve os PWMs. A telemetria sai a **25 Hz** pelas duas portas.

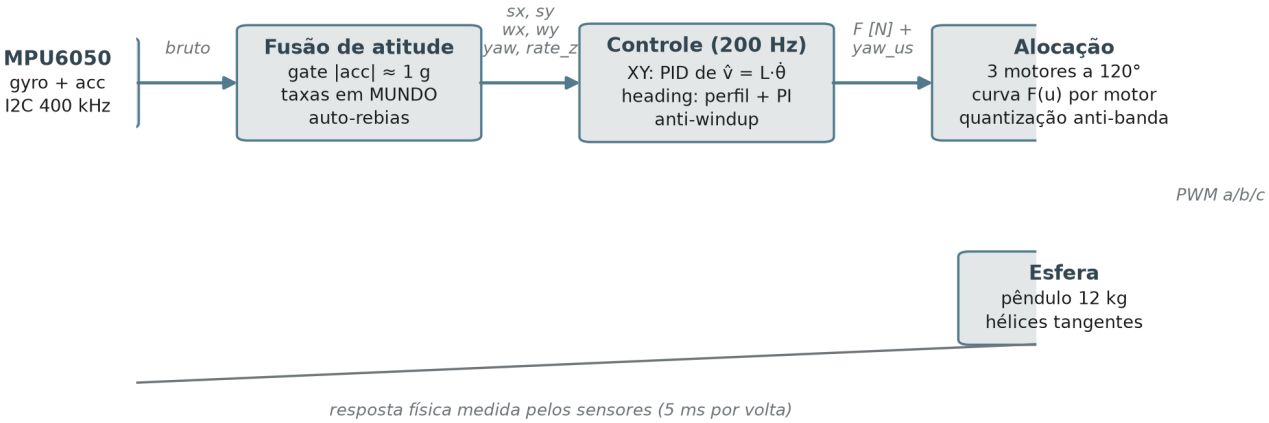


Figura 4.1 — Blocos do loop fechado (Apêndice C detalha cada seta).

4.1 Fusão de atitude (inclinação e heading)

• **Inclinação (sx, sy)**: fusão complementar — integra as taxas do giroscópio e corrige com a inclinação absoluta do acelerômetro por atan2. A correção do acelerômetro só entra com |acc| entre 0,8 e 1,2 g (gate gravimétrico): com os motores ligados, a vibração não contamina o ângulo.

• **Taxas em mundo**: as taxas de inclinação usadas pelo controle são compensadas pela rotação de yaw (teorema do transporte, constante COMP_YAW_SIGN). Sem isso, girar a esfera e amortecer ao mesmo tempo amortece na direção errada.

• **Heading**: integração direta da taxa do giroscópio, sem zona morta no integrador. Deriva alguns graus por minuto — normal sem referência absoluta; o apontamento fino vem da visão (Cap. 9) ou do re-zero periódico.

• **Auto-rebias**: com motores em neutro e taxas próximas de zero por mais de 3 s, o bias do giroscópio se ajusta sozinho (constante de tempo ~50 s). Corrige a deriva térmica sem precisar recalibrar.

4.2 Modelo por motor e alocação

Cada motor guarda sua calibração: direção de empuxo no corpo (t_x, t_y), sinal de yaw (±1), ganhos da curva frente/ré (kf = N/µs^2) e zona morta (µs). A curva é quadrática: $F(u) = kf \cdot (u - dz)^2$ acima da zona morta. A alocação converte a força desejada no corpo em µs por motor: $f_i = (2/3) \cdot (t_i \cdot F)$ para a força, mais $yaw_sign_i \times yaw_us$ para o heading. Se algum motor exceder a reserva de 490 µs, todos escalam proporcionalmente (a direção se preserva). Nada assume que os três motores foram montados iguais — o wizard mede cada um (Cap. 8).

4.3 Loop de translação (X e Y): amortecimento

PID sobre a velocidade estimada $\hat{v} = L \cdot \dot{\theta}$, com setpoint zero e anti-windup por clamping (a integral só cresce quando não empurra a saída contra o limite). A restauração gravitacional devolve a esfera ao vertical; o controlador drena a energia de oscilação. Histerese anti-chatter: comando ínfimo vira neutro.

4.4 Loop de heading: cascata com frenagem

Erro de apontamento (da visão quando fresca, senão do setpoint interno) → **perfil de taxa** limitado com frenagem embutida — a taxa pedida é o mínimo entre $kp \cdot |erro|$, $\sqrt{(2 \cdot a \cdot |erro|)}$ e o limite — → PI de taxa em µs/motor com anti-windup. O perfil garante que a distância de frenagem nunca exceda o erro restante: degraus grandes chegam sem sobressinal. O feedforward torcional existe no código, mas vale 0 (cabo não devolve o heading).

4.5 Segurança

- Loops de controle **desligados no boot**: 'y' liga o heading, 'p' liga o anti-balanço.
- Faixa de PWM 1500 ± 500 µs (limite físico do ESC, calibrado).
- 'k' = parada geral: loops off + ESCs em neutro + polia e pinça param.
- Fins de curso travam a pinça nos extremos; calibração da MPU aborta se a esfera estiver em movimento (gate de quietude).

5. Referência de comandos

Todos os comandos funcionam **igualmente nas duas portas** — USB (só na bancada de flash) e Serial2 via Rasp (a porta de campo; do terminal da Rasp, use nc localhost 9999 através do agente). Comandos de um caractere executam imediatamente (sem Enter); comandos em linha terminam com Enter. Selos: **[OBRIGATÓRIO por sessão]** obrigatório por sessão · **[condicional]** condicional (depende do evento) · **[opcional]** opcional.

5.1 Comandos de caractere (sem Enter)

Cmd	Selo	O que faz / efeitos colaterais
z	**[OBRIGATÓRIO por sessão]**	Recalibra MPU (gyro + zero gravimétrico) com gate de quietude (aborta se a esfera se mover). **Zera o heading e as estimativas**. Fazer a cada ligar e após manusear a esfera.
L	**[OBRIGATÓRIO por sessão]**	Comprimento do cabo da sessão em m (0,5–6,0). Ex.: L1.5. Obrigatório se L ≠ 1 m — entra na estimativa de velocidade $\hat{v} = L \cdot \dot{\theta}$.
p / b	**[OBRIGATÓRIO por sessão]**	Liga/desliga o **anti-balanço** (X/Y). Ao desligar, zera força e integrais. Começa DESLIGADO no boot.
y	**[OBRIGATÓRIO por sessão]**	Liga/desliga o **heading**. Ao desligar, zera a integral da taxa. Começa DESLIGADO no boot.
t	**[condicional]**	Calibra L pelo período natural (precisa ter balançado a esfera livre antes, com anti-balanço OFF). Usa $L = g \cdot (T/2\pi)^2$. Verificar contra o L real.
m	**[condicional]**	Teste sequencial dos 3 motores (±120 µs, 2 s cada sentido). Desliga os loops durante (~18 s). Rodar após montar/mexer na fiação.
0	**[condicional]**	ψ desejado = 0 e zera a integral da taxa.
k	**[opcional]**	PARADA GERAL: loops off + ESCs em 1500 + polia e pinça param.
w / s / x	**[opcional]**	Pinça sobe / desce / para (bloqueada pelos fins de curso).
a / d / c	**[opcional]**	Polia gira sentido A / D / para (varre o comprimento do cabo).
e	**[opcional]**	Lê o ângulo do AS5600 (polia) na USB.

Tabela 5.1 — Comandos de caractere.

5.2 Comandos em linha (com Enter)

Comando	Selo	O que faz
	[condicional]	Degrau **relativo** de heading em graus. Ex.: 90 gira +90° a partir da posição atual; -45 gira −45°. Re-zera o ângulo integrado.
G<índice>	**[opcional]**	Ajusta um ganho ao vivo (tabela 5.3). Ex.: G0 50. Aplica imediatamente, sem reiniciar.
R	**[opcional]**	Devolve os 8 ganhos no formato GAIN,índice,nome,valor.
V	**[condicional]**	Visão (da Rasp): bearing do cabo da linha em graus no referencial do corpo + distância D em m. Válido por 500 ms; enquanto fresco, o bearing É o erro de apontamento. O D vai para a telemetria.
W <µs>	**[condicional]**	Pulso do wizard: desliga os loops e pulsa o motor 0/1/2 com o offset de µs pela duração. Usado pelo wizard.py.
M	**[condicional]**	Calibração por motor em RAM (enviada pelo wizard com --push). Ex.: M0 1 0 +1 8.15e-05 4.89e-05 30.

Tabela 5.2 — Comandos em linha.

5.3 Índices de ganho (G e R)

Índice	Ganho	Unidade	Default	O que controla
G0	vel_kp	N por (m/s)	45	rigidez do amortecimento XY
G1	vel_ki	N/m	2,2	mola lenta de posição (pequeno de propósito)
G2	vel_kd	N·s/m	0	amortecimento adicional do erro
G3	head_angle_kp	(rad/s)/rad	2,5	agressividade do apontamento
G4	head_decel	rad/s^2	1,5	frenagem do perfil (≥ = mais rápido, risco de sobressinal)
G5	head_rate_limit	rad/s	2,5	velocidade máxima de curva
G6	head_rate_kp_us	µs/(rad/s)	900	rigidez da malha de taxa
G7	head_rate_ki_us	µs/rad	2000	zera erro de taxa em regime

Tabela 5.3 — Ganhos endereçáveis (os mesmos nomes existem no simulador e no twin).

5.4 Telemetria (25 Hz, as duas portas)

```
TEL,t_ms,yaw,bearing,yaw_des,rate_z,sx,sy,wx,wy,pwm_a,pwm_b,pwm_c,L,D,flags
TEL,123456,90.12,0.00,90.00,1.50,1.234,-0.567,4.500,-2.100,1620,1380,1500,1.00,2.35,15
```

Ângulos e taxas em graus/°s; PWM em µs; L e D em m (D = −1 sem visão). **flags** (soma de bits): 1 = yaw ligado, 2 = anti-balanço ligado, 4 = MPU calibrada, 8 = visão fresca, 16 = wizard/pulso ativo.

5.5 Teclas do simulador interativo

Tecla	Ação
setas (segurar)	força externa contínua em ±X/±Y do mundo
Q / E (segurar)	torque externo de ±0,05 N·m no heading
1 / 2	liga/desliga controle pendular (XY) / heading
Z / X	ψ desejado −15° / +15°
G, depois + / −	seleciona item (força ext. e os 8 ganhos) e ajusta em passos de 10%
R / P / Esc	reset à condição inicial / pausa / sair
arrastar mouse	gira a vista 3D

Tabela 5.4 — Teclado do modo interativo (python main.py --interactive).

5.6 Teclas do twin — semântica dupla

No twin (python -m twin), as mesmas teclas atuam na simulação **e** no hardware: **1/2** enviam p/y ao STM (e alternam os loops no simulador); **Z/X** enviam degraus relativos de heading; **+/−** ajustam o ganho na simulação **e o empurram ao STM** pelo comando G correspondente; **K** dispara a parada geral nos dois lados. Setas e Q/E continuam só na simulação (forças externas preditivas, para comparar real vs. simulado).

5.7 Linhas de comando (PC)

```
python main.py [--scenario ...] [--interactive] [--compare] [--no-anim]
                [--mass 12] [--length 1.0] [--thrust 18] [--inertia 1.728]
                [--calib calib.json] [--save saida.gif]
python -m twin  --sim | --serial COM7 | --tcp 192.168.0.50[:9999]  [--scenario ...]
python wizard.py --sim | --serial COM7 | --tcp HOST [--out calib.json] [--push]
python3 Raspberry/main.py      # na Rasp, a partir da raiz do repo (visão + ponte)
python -m pytest tests/ -q     # 48 testes (+1 do bearing roda com cv2 instalado)
```

6. Procedimento de operação

6.1 Sequência de bancada (com frequência/gatilho de cada etapa)

#	Etapa	Comando/procedimento	Quando fazer (frequência/gatilho)
1	Flash + smoke test	compilar e gravar firmware/main.cpp no PC (USB), abrir a serial e conferir banner + boot com loops OFF. Sequência completa de bring-up em etapas (ESCs desligados até a etapa 5): docs/BRINGUP.md	uma vez por versão do firmware (exige buscar o STM)
2	Teste de motores	'm' — confirmar a↔PA0/b↔PA15/c↔PA1 e os sentidos de giro	após montar/mexer na fiação dos motores
3	Eixos do MPU	incluir a esfera em direção conhecida: sx/sy da telemetria devem responder nos eixos certos; girar a esfera inclinada à mão: sx/sy NÃO podem girar junto (senão, inverter COMP_YAW_SIGN no firmware)	após remontar o sensor ou trocar a orientação do chip
4	Wizard por motor	python wizard.py --serial COM7 --push (na bancada) ou --tcp / --tcp localhost:9999 na Rasp (Cap. 8)	**obrigatório na 1ª sessão** e após qualquer troca de motor/ESC/hélice; recomendado a cada grande revisão
5	Calibração 'z'	esfera parada no prumo, comando z	**todo boot** e após manusear/transportar a esfera
6	L da sessão	L1.5 (manual) ou 't' após balanço livre	sempre que trocar o comprimento do cabo; 't' também serve de verificação
7	Sessão de controle	p (anti-balanço), y (heading), Z/X ou para o heading; ganhos com G se precisar sintonizar	a cada operação
8	Encerrar	'k' (parada geral) antes de desligar ou tocar na esfera	sempre, por segurança

Tabela 6.1 — Fluxo de bancada.

6.2 Fluxo comentado da primeira sessão

```
# 1. flash + abrir serial (115200) — loops DESLIGADOS no boot
m          # confere os 3 motores e os sentidos (~18 s)
# 2. wizard (PC): mede cada motor e envia a calibração
python wizard.py --serial COM7 --push
# 3. esfera parada no prumo:
z          # calibra MPU (aborta se a esfera se mover)
L1.0       # comprimento do cabo da sessão
# 4. liga os loops e opera:
p          # anti-balanço (amortecimento XY)
y          # heading
45         # gira +45 graus a partir da posição atual
# 5. sintonia fina pelo twin (opcional): G0 50 / R para ler
k          # parada geral ao terminar
```

6.3 Roadmap: fase drone (não implementado)

> **Futuro — fora do escopo atual**
> Com a esfera suspensa por drone, o pivô passa a se mover. O modelo já tem pivot_accel (aceleração do pivô como distúrbio) e o desenho recomendado é: manobras suaves do drone (ou input shaping), feedforward da aceleração do pivô no controlador e o loop de visão fechando a posição sobre a linha. Nada disso está implementado hoje.

7. Calibrações: automático vs. input do usuário

7.1 Totalmente automático (sem ação do usuário)

Calibração	Como funciona
Auto-rebias do giroscópio	com ESCs em neutro e taxas ~0 por 3 s, o bias de taxa se ajusta lentamente (constante ~50 s). Corrige deriva térmica em operação.
Gate gravimétrico	a correção do acelerômetro só entra com |acc| entre 0,8 e 1,2 g; vibração dos motores é rejeitada sozinha.
Gate de quietude da calibração	'z' se auto-nega se a esfera estiver em movimento.
Rescale de ganhos por inércia	no simulador, --inertia com o I medido rescala o loop de taxa do heading automaticamente (regra kp·k_u/I).

Tabela 7.1 — Calibrações automáticas.

7.2 Semiautomáticas (você dispara, o sistema confere)

Calibração	Comando	Frequência/gatilho	O que mede
MPU (gyro + zero gravimétrico)	z	todo boot / após manusear	bias do gyro e ângulo de repouso; exige esfera parada
L pelo período	t	após trocar o cabo / verificação	$L = g\cdot(T/2\pi)^2$ do período de balanço livre
Modelo por motor (wizard)	python wizard.py --push	1ª sessão; troca de motor/ESC/hélice	direção de empuxo, sinal de yaw, curva frente/ré, zona morta

Tabela 7.2 — Calibrações semiautomáticas.

7.3 Input do usuário

Parâmetro	Como informar	Notas
L da sessão	L na serial (ou --length no simulador)	0,5–6,0 m; entra em $\hat{v} = L\cdot\dot{\theta}$ e na telemetria
Heading desejado	(degrau relativo) ou Z/X no twin	sem visão, o referencial deriva — re-zero periódico
Ganhos	G<índice> (ou teclas G/+/− no twin)	R devolve os atuais; defaults já sintonizados
Parâmetros físicos (simulador)	--mass, --thrust, --inertia, --calib	firmware: editar constantes do topo do main.cpp

Tabela 7.3 — Parâmetros de entrada do usuário.

7.4 Ainda não calibrado (a medir na bancada)

Parâmetro	Valor atual	Procedimento proposto
Atraso de atuação τ (ESC + hélice)	120 ms (estimativa minha)	mesmo degrau de pulso no hardware e no simulador, varrer τ do modelo até as respostas se sobreporem; afeta o wizard (~20% no kf) e a margem de estabilidade dos ganhos altos
Faixa de travamento do ESC	~±60 µs (provisório)	modelada e mitigada no caminho de controle (Cap. 10.5); medir por ESC na bancada e refinar o default (rotina do wizard planejada)
Banda de não-atuação	não implementada	decidir a tolerância em graus/fora-de-band; alimenta a política proposta no Cap. 10

Tabela 7.4 — Pendências de calibração.

8. O wizard de identificação por motor

Nada garante que os três motores foram soldados ou montados iguais: ESC com fase trocada inverte o sentido, hélice montada espelhada inverte ou degrada o empuxo, e cada conjunto tem sua própria curva. O wizard **mede** cada motor provocando pulsos e lendo a resposta pela própria telemetria — sem balança e sem hardware extra.

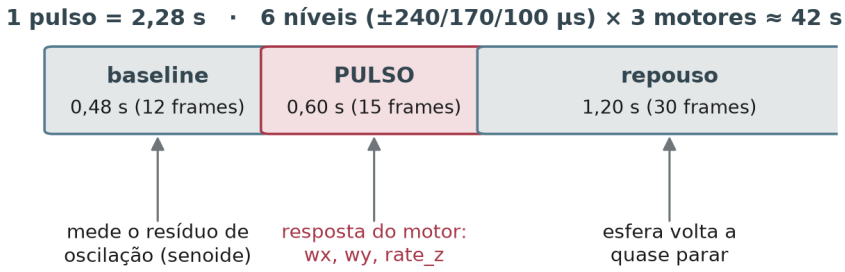
8.1 Procedimento (visão do operador)

• Esfera pendurada e **livre para balançar** (vai balançar — é assim que se mede força); sem vento; L correto no firmware (o wizard o lê da telemetria).

• Rodar: **python wizard.py --serial COM7 --push** (ou --sim para treinar contra o STM simulado, ou --tcp HOST pela Rasp).

• Ele desliga os loops, aplica 6 pulsos por motor (+240/+170/+100/−100/−170/−240 µs, 600 ms cada, com repouso entre eles) e coleta a telemetria.

• No fim: imprime o relatório por motor, salva **calib.json** e, com --push, envia a calibração ao STM (comando M, vale em RAM).



Comando W<motor> <µs> <ms>: loops desligados durante o pulso; telemetria TEL segue a 25 Hz — é por ela que o wizard mede.

Figura 8.1 — Cronograma de um pulso (repetido 6× por motor, 3 motores ≈ 42 s no total).

8.2 A matemática, em quatro blocos

8.2.1 Subtração do resíduo de oscilação

O pêndulo quase não amortece ($\zeta \approx 0{,}02$): a oscilação do pulso anterior (~16°/s) é 8× a resposta de um pulso reverso. Na janela de baseline o wizard ajusta, por mínimos quadrados, um senoide $w(t) = a\cdot\sin(\omega t) + b\cdot\cos(\omega t) + c$ com $\omega = \sqrt{(g/L)}$ **conhecida** (não ajustada), no referencial do mundo (eixo de balanço fixo), e extrapola/subtrai esse resíduo durante o pulso.

8.2.2 Transporte para o corpo, amostra a amostra

O pulso gira a esfera ~15° em 600 ms. Cada amostra (wx, wy) é girada por $R_z(-\psi(t))$ **antes** da média; medir a direção no mundo com um yaw médio borraria o resultado.

8.2.3 Força a partir da resposta do pêndulo

Para força constante F durante T a partir do repouso: mean(θ̇) = F·(1−cos(ωT))/(m·L·ω^2·T). Invertendo cada pulso dá um ponto (μs, F). Como a força real sobe com atraso de 1ª ordem (τ = 120 ms), a resposta vem multiplicada por g < 1 — o wizard divide pelo fator fechado lag_mean_factor(T, τ, ω); sem isso o kf sairia ~36% baixo. Viés restante: se o τ real diferir do assumido, o erro fica em ~±20% (Cap. 7.4).

8.2.4 Regressão da curva

A curva F = kf·(u − dz)^2 lineariza em √F = a·u + b. Regressão nos 3 pontos de cada sentido dá kf = a^2 e dz = −b/a. Frente e ré são ajustados separadamente (hélice de passo fixo é menos eficiente em ré). Falhas de fit (pontos insuficientes, inclinação negativa, dz implausível) preservam o default daquele campo.

8.3 Saída e validação

calib.json usa o mesmo formato do simulador (python main.py --calib calib.json) — a partir daí simulador, twin e firmware falam da mesma calibração. O teste test_wizard_recovers_perturbed_calibration prova o procedimento: com um motor sabotado (rotacionado 80°, yaw invertido, curva 1,5×), o wizard recupera a direção com erro ≤ 2–7°, sinal de yaw exato e curva dentro da tolerância.

8.4 Limites

- Mede empuxo relativo à verdade de m e L (usa 12 kg e o L da telemetria).
- Níveis de pulso moderados de propósito: pulsos maiores deflectem a esfera para fora do regime de pequenos ângulos e estragam a identificação.
- Não mede braço de torque nem massa — entram como constantes conhecidas.
- A faixa de travamento do ESC (Cap. 11) pode distorcer o dz medido perto do neutro — caracterizar na bancada.

9. Digital Twin

9.1 Transportes e como escolher

Transporte	Comando	Quando usar
Serial USB direta	python -m twin --serial COM7	bancada de flash/testes: STM na USB do PC, zero rede, zero configuração
TCP via ponte do app de visão	python -m twin --tcp 192.168.0.50[:9999]	operação real: o PC alcança a Rasp na rede (LAN, hotspot, túnel) e a ponte embutida no app de visão serve a serial USB do STM
Túnel SSH	ssh -L 9999:localhost:9999 pi@ e depois --tcp 127.0.0.1:9999	quando o PC não alcança a Rasp por IP direto
Terminal da própria Rasp	nc localhost 9999 (teclado/mouse/monitor da Rasp)	sem PC nenhum: a mesma linha de comandos, através da ponte do app de visão
STM simulado	python -m twin --sim	desenvolver/treinar sem hardware; mesma planta, atuadores, controlador e protocolo

Tabela 9.1 — Transportes do twin.

9.2 Deploy na Raspberry Pi

```
git clone https://github.com/iNeewbie/esfera.git && cd esfera
pip3 install -r Raspberry/requirements.txt   # opencv-CONTRIB (WLS), pyserial, gpiozero
python3 Raspberry/main.py                    # a partir da raiz do repo
```

O app de visão é o **único dono da serial USB do STM** (auto-scan de /dev/ttyUSB* e /dev/ttyACM*, 115200). No boot ele pulsa o reset físico do STM (GPIO18) para um estado limpo, abre a serial e sobe a ponte TCP na porta 9999. Toda linha do STM vai aos clientes TCP (twin, wizard, nc); comandos de qualquer cliente voltam ao STM. No terminal da Rasp não se abre screen/minicom direto — o caminho é nc localhost 9999. O wizard também roda na própria Rasp: python wizard.py --tcp localhost:9999 --push.

Sobre o remote desktop sem HDMI (VNC/RDP headless): ótimo para a janela do app de visão e terminais; rode porém a **janela do twin no PC** via --tcp, pois animação matplotlib sobre remote desktop fica arrastada. Nota de segurança: quem controla a Rasp controla a esfera; proteja o SSH (chave/senha forte) e lembre que o k (parada geral) funciona de qualquer cliente conectado.

9.3 A janela do twin

• **Esfera fantasma** (cinza, tracejada): estado REAL da telemetria — inclinação, heading e empuxos recalculados dos PWMs medidos — sobreposta à esfera simulada.

• **Linha de transmissão** marrom a distância D abaixo da esfera, orientada pelo heading + bearing, quando a visão estiver enviando D.

• **Curvas real (tracejado) vs. simulada (sólido)**: inclinação, velocidade, heading e PWM dos ESCs; HUD com conexão, estado dos loops, L e D.

• **Comandos aos dois lados**: 1/2 (loops), Z/X (heading), K (parada geral) e ganhos +/− com push ao STM. Setas e Q/E ficam só na simulação (perturbações preditivas).

9.4 O contrato da visão (IMPLEMENTADO no app da Rasp)

As câmeras ficam fixas ao sensor (giram com a esfera) e olham **para baixo**, para o cabo da linha. A cada frame com linha detectada (tecla **v** liga/desliga o envio), o app envia ao STM: **V<bearing> <D>** — o bearing do cabo no referencial da câmera, **com sinal**, normalizado em (−90°, 90°], mais a distância estereoscópica D (−1 no modo mono). O bearing É o erro de apontamento (imune à deriva do gyro, por construção — câmeras fixas ao sensor); o STM o usa como erro direto enquanto fresco (< 500 ms). A distância D viaja na telemetria: o twin desenha a linha abaixo da esfera e o operador a usa na aproximação.

> **Pendência de bancada: sentido do bearing**
> O SINAL do bearing é validação de bancada (etapa 8 do BRINGUP): a cadeia visão→yaw nunca fechou no firmware antigo (o PID de heading estava morto), então não existe convenção já provada. Se a esfera girar para o lado errado ao alinhar com o alvo, inverta o sinal no envio — uma linha em Raspberry/main.py.

10. Configurações avançadas

10.1 Constantes do firmware (topo de firmware/main.cpp)

Constante	Default	Efeito / quando mexer
PWM_LIMIT_US	500	faixa de PWM (1500 ± x). Reduzir volta ao modo conservador (envelope cai com o quadrado)
SWING_RESERVE_US / YAW_RESERVE_US	490 / 300	orçamento de µs do anti-balanço e do heading; a soma nunca deve passar da faixa
F_MAX_500 / REV_RATIO / DB_US	18 N / 0,60 / 30 µs	curva provisória — o wizard substitui por motor (comando M)
ESC_LATCH_US / LATCH_MARGIN_US	60 / 15 µs	faixa de travamento direcional do ESC + margem: no caminho de controle, o menor µs não-nulo nasce fora da faixa e comando sub-piso vira neutro (mitigação ativa; o caminho W do wizard não quantiza)
COMP_YAW_SIGN	+1	sinal da compensação de yaw na fusão; inverter se o teste do Cap. 6 (etapa 3) falhar
ALPHA_FUSAO	0,88	peso do giroscópio na fusão (dt = 5 ms); aumentar = mais confiança no gyro, menos no acc
ACC_GATE_LO / HI	0,8 / 1,2 g	janela de aceitação do acelerômetro; estreitar se a vibração passar
LEAD_S / lag_tau	— / 0,12 s	atraso de atuação do MODELO (actuators.py no Python; o firmware não modela — o hardware o tem de verdade). Substituir pelo valor medido (Cap. 7.4)

Tabela 10.1 — Principais constantes.

10.2 Orçamento de µs por motor

Cada motor recebe a soma do anti-balanço (até 490 µs) e do heading (até 300 µs, com o sinal calibrado), e o total é clipado em ±500 µs. Em manobras com os dois loops ativos, o heading pode ser espremido pela

saturação do balanço — o escalonamento proporcional preserva a direção da força, mas o torque de heading disponível cai. Se precisar de mais autoridade de heading durante curvas, reduza SWING_RESERVE_US.

10.3 Rescale de ganhos por inércia

A resposta do heading depende de kp·k_u/I. Medindo o I real (oscilação livre no heading), passe --inertia no simulador: os ganhos de taxa (G6/G7) rescalam por I/I_ref e o perfil de frenagem por I_ref/I, com I_ref = 1,728 kg·m^2. No firmware, multiplique head_rate_kp_us e head_rate_ki_us por I_real/1,728 e divida head_decel pelo mesmo fator.

10.4 Limites de segurança vigentes

- Loops desligados no boot; 'k' sempre disponível e de efeito imediato.
- Faixa de PWM limitada à calibração do ESC (1000–2000 µs).
- Calibração da MPU aborta sem quietude; telemetria segue mesmo com sensor falho (flags).
- Fins de curso e deadzones dos mecanismos independem do controle.

10.5 PROPOSTA (não implementado): política de perto-do-neutro

> **Mitigação ativa + proposta de gerenciador — aguardando caracterização em bancada**
> Problema conhecido em bancada: PWMs pequenos (relato: ~±60 µs) podem travar o motor naquele sentido até uma reversão; e em regime permanente os motores ficam alternando de lado. Mitigação mínima JÁ ATIVA (paridade Python/firmware): o menor µs não-nulo comandado nasce FORA da faixa de travamento (latch + 15 µs de margem) — comandos sub-piso viram neutro, tanto no caminho de força (gate por motor) quanto no µs final (quantização anti-banda de swing+yaw); o caminho do wizard não quantiza de propósito, para manter o modelo do ESC exposto e medível. Ainda PROPOSTO (não implementado — decidir com os números medidos): (1) caracterizar a faixa de travamento por ESC na bancada; (2) gerenciador de perto-do-neutro na camada de atuador — banda morta com histerese de Schmitt na norma da força (ativa > ~0,5 N, inativa < ~0,25 N), integração condicional e permanência mínima de sentido (~200 ms); (3) chute de partida curto fora da faixa antes de assumir um sentido a partir do neutro. Custo: droop residual de ~0,25–0,5° dentro da banda (absorvido pelo loop de visão).

11. Solução de problemas

Sintoma	Causa provável	Ação
Motor travou após PWM pequeno e só volta ao inverter o sentido	travamento direcional do ESC (~±60 µs) — mitigado no caminho de controle (gate + quantização anti-banda, Cap. 10.5); comandos brutos (wizard) ainda exercem a faixa	reverter o sentido brevemente com µs fora da zona morta; médio prazo: caracterização por ESC + gerenciador completo do Cap. 10.5
Heading deriva com o tempo	deriva do giroscópio (sem referência absoluta)	'z' com a esfera parada; em operação longa usar a visão (V) como referência
Esfera amortece na direção errada quando gira	COMP_YAW_SIGN invertido ou eixos do MPU trocados	refazer o teste de eixos (Cap. 6, etapa 3) e inverter a constante se necessário
Motores alternam de lado em regime permanente	comando ruído em torno de zero + alocação redistribuindo	aceitável dentro da banda; se incomodar, a política do Cap. 10.5 resolve
Telemetria não aparece	serial/baud errados; app de visão parado na Rasp; porta ocupada por outro processo	conferir 115200 e a porta; na Rasp, o app de visão precisa estar rodando (ele é o dono da serial); usar nc localhost 9999 — nunca abrir a serial direto
Esfera gira para o lado errado ao alinhar com a linha	sinal do bearing invertido (convenção nunca validada no firmware antigo)	inverter o sinal no envio (uma linha em Raspberry/main.py); ver Cap. 9.4
[ERRO] MPU em movimento ao calibrar	gate de quietude detectou movimento	esperar a esfera parar de verdade e repetir 'z'
AS5600 não encontrado	fiacao/barramento I2C	conferir fiação; 'e' tenta de novo; destravar barramento reiniciando se travou
Pinça não sobe/desce	fim de curso acionado	o firmware bloqueia o sentido proibido ('[BLOQUEADO]'); afastar do extremo
Comando digitado não faz nada	caracteres fora da whitelist viram linha; linha desconhecida é ignorada	conferir a sintaxe no Cap. 5; números e linhas precisam de Enter
Controle oscila com ganhos altos	atraso real do ESC maior que o modelado	medir τ (Cap. 7.4) e reduzir vel_kp / head_decel até estabilizar

Tabela 11.1 — Sintoma → causa → ação.

Apêndice A — Cartão de referência rápida

Uma página para imprimir e colar na bancada. Onde digitar: na bancada, na serial USB; em campo, nc localhost 9999 no terminal da Rasp, ou pelo twin no PC (--tcp).

Comando	Ação
z	calibrar MPU (esfera parada!) — a cada boot **[OBRIGATÓRIO por sessão]**
L1.0	comprimento do cabo da sessão (m)
p	anti-balanço ON/OFF (X/Y)
y	heading ON/OFF
90 / -45	girar heading (degrau relativo, graus)
0	ψ desejado = 0
m	teste dos 3 motores (~18 s, desliga loops)
t	calibrar L pelo período de balanço livre
G0 50 / R	ajustar ganho / ler os 8 ganhos
k	PARADA GERAL
w/s/x · a/d/c · e	pinça sobe/para/desce · polia A/D/para · ler AS5600
W1 -150 600	pulso do wizard (motor, µs, ms)
V12.5 3.2	visão: bearing (°) + distância D (m)
M0 ...	calibração por motor (via wizard --push)

Tabela A.1 — Comandos seriais.

Tecla (twin/interativo)	Ação
setas / Q E	força externa XY / torque no heading (só simulação)
1 / 2	loops XY / heading — no twin vão também ao hardware
Z / X	ψ desejado −15°/+15° — no twin também
G, + / −	ganhos ao vivo — no twin com push ao STM
K (twin) / k (serial)	parada geral
R / P / Esc	reset / pausa / sair

Tabela A.2 — Teclado.

Apêndice B — Fórmulas do modelo

- **Curva do motor**: $F(u) = kf \cdot (u - dz)^2 \cdot \text{sign}(u)$, com kf própria para frente e ré; inversa $u(F) = \text{sign}(F) \cdot (dz + \sqrt{(|F|/kf)})$.
- **Alocação**: $f_i = (2/3) \cdot (t_i \cdot F_corpo) + yaw_sign_i \cdot yaw_us$; força no mundo $= R_z(\psi) \cdot \Sigma\, f_i \cdot t_i$; torque $= R \cdot \Sigma\, yaw_sign_i \cdot f_i$.
- **Pêndulo (linearizado)**: $\ddot{x} = -(g/L) \cdot x + F_x/m$ (idem y); $\omega = \sqrt{(g/L)}$.
- **Heading**: $I_\psi \cdot \ddot{\psi} = \tau$; cascata erro → taxa pedida $\min(k_p \cdot |e|, \sqrt{(2a|e|)}, \text{limite})$ → PI de taxa em µs.

• **Resposta do pêndulo a um pulso** (wizard): mean(θ) = F·(1−cos(ωT))/(m·L·ω^2·T), corrigida pelo fator do atraso g(τ).

• **L calibrado pelo período**: L = g·$(T/2\pi)^2$.

Apêndice C — O loop fechado em detalhe

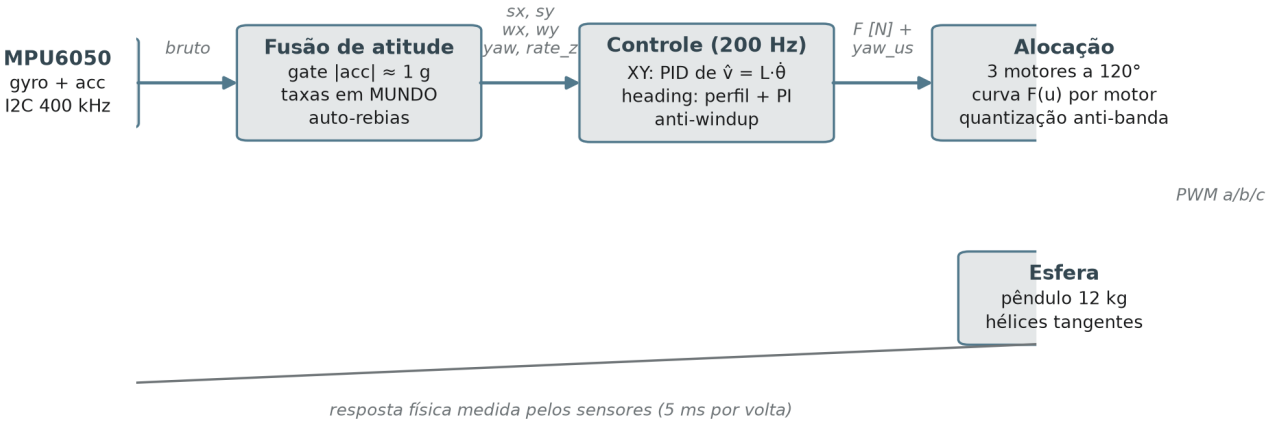


Figura C.1 — Blocos do loop de controle (ver Cap. 4).

Seta	Sinal	De → para	Observação
1	gyro/acc (14 bytes, I2C 400 kHz)	MPU → fusão	leitura a cada volta da malha
2	sx, sy, wx, wy, yaw, rate_z	fusão → PIDs	ângulos e taxas no MUNDO
3	F_x, F_y [N]	PID XY → alocação	força desejada no mundo
4	yaw_us	PI de taxa → alocação	µs por motor (sinal calibrado)
5	PWM a/b/c	alocação → ESCs	clip na faixa 1000–2000 µs
6	resposta física	esfera → MPU	fecha o laço; lida na próxima volta (5 ms)

Tabela C.1 — As setas da Figura C.1.